Design and Implementation of a Local Retrieval-Augmented Generation Chatbot for PDF Question Answering

Bhaireswarudu Majjari

Independent Project Technical Report

Status note: *This is a research-style technical report, not a peer-reviewed or formally accepted publication. It reports no unmeasured performance results.*

Abstract

Large language models can produce fluent answers, but their responses may not reflect the contents of a user's own documents. Retrieval-Augmented Generation (RAG) addresses this limitation by retrieving relevant text from a knowledge source and providing it to a language model as context. This paper describes the design and implementation of a local AI chatbot that answers questions about uploaded PDF documents. The application combines a React frontend, a FastAPI backend, PDF text extraction with PyMuPDF, semantic embeddings using the Sentence Transformers model all-MiniLM-L6-v2, vector similarity search with FAISS, and local language-model inference through Ollama using llama3.2:3b. The system exposes endpoints for health checks, general chat, PDF upload, and document-grounded question answering. The implementation demonstrates a practical local RAG workflow and provides a foundation for further evaluation. No formal benchmark results are claimed in this paper; retrieval accuracy, answer faithfulness, latency, and performance across different document types remain subjects for systematic future evaluation.

Keywords

Retrieval-Augmented Generation; RAG; PDF question answering; local AI; large language models; embeddings; FAISS; FastAPI; Ollama; React.

1. Introduction

Users often need to ask specific questions about long documents such as reports, manuals, and study materials. Reading every page manually can be time-consuming. A conversational interface can make these documents easier to explore, but a language model used without document context may produce answers that are incomplete or unsupported by the source.

Retrieval-Augmented Generation combines information retrieval with text generation. A document is processed into manageable text segments, those segments are represented as vectors, and relevant segments are retrieved when a question is asked. The retrieved content is then supplied to a language model to help generate a response grounded in the document.

This project implements a local PDF question-answering chatbot. Its primary objective is to demonstrate an end-to-end RAG workflow using an accessible web interface and locally hosted language-model inference.

2. Problem Statement and Objectives

The problem addressed is how to provide a simple conversational interface for asking questions about an uploaded PDF, while retrieving relevant document content to support the answer.

The implementation objectives are: (1) provide a web interface for interacting with the chatbot; (2) accept PDF uploads and extract their text; (3) represent document text as embeddings; (4) retrieve relevant text using vector similarity search; (5) pass retrieved context to a locally served language model; and (6) expose backend endpoints for chat, document upload, and document-based questions.

3. System Architecture

The application follows a frontend-backend architecture. The React frontend provides the user interface. The FastAPI backend coordinates chat requests, PDF processing, retrieval, and calls to the local Ollama service. For PDF questions, extracted text is embedded using Sentence Transformers and indexed for similarity search with FAISS. Retrieved text is used as context for the language model.

The main components are:

• Frontend: React application served during development by Vite.

• API layer: FastAPI endpoints for health checks, general chat, PDF upload, and PDF chat.

• PDF processing: PyMuPDF extracts text from uploaded PDF files.

• Embedding model: all-MiniLM-L6-v2 from Sentence Transformers converts text segments into vectors.

• Vector retrieval: FAISS searches the vector index for relevant text segments.

• Language model: Ollama serves the llama3.2:3b model locally and generates responses using the supplied context.

The backend was configured for local development on port 8001, with the frontend using Vite on port 5173. These are development settings, not production deployment recommendations.

4. Methodology

4.1 PDF ingestion. A user uploads a PDF through the application. The backend uses PyMuPDF to extract text from the document. The extracted text is prepared for downstream retrieval. Scanned PDFs that contain only page images may require optical character recognition; OCR support is not claimed here.

4.2 Text representation. Text segments are transformed into dense vector embeddings using the all-MiniLM-L6-v2 Sentence Transformers model. Embeddings allow text passages to be compared by semantic similarity rather than relying only on exact keyword matches.

4.3 Vector indexing and retrieval. FAISS is used to search the embedding index for passages relevant to a user's question. The retrieved passages provide document context for the answer-generation stage. The quality of retrieval depends on factors such as document structure, segmentation, embedding behavior, and the question asked.

4.4 Answer generation. The backend sends the user question and retrieved document context to the locally served Ollama model llama3.2:3b. The model generates a natural-language response based on the supplied context. Retrieval can help ground an answer, but it does not guarantee correctness; the model may still misunderstand the source or generate unsupported statements.

4.5 API and user interaction. The backend includes endpoints for health checking, general chat, PDF upload, and PDF-based chat. The frontend lets the user interact with the chatbot and submit a PDF for document question answering. During development, the API documentation was available through FastAPI's interactive documentation page.

5. Implementation and Technology Stack

The system uses Python for backend orchestration and AI-related processing, FastAPI for HTTP endpoints, PyMuPDF for PDF text extraction, Sentence Transformers for embeddings, FAISS for vector search, and Ollama for local language-model inference. The user interface is implemented in React and run with Vite during development.

The selected stack provides a practical demonstration of how a document-grounded assistant can be assembled from common components. Local inference can reduce reliance on a hosted LLM API for generation, but it does not by itself guarantee complete privacy: file handling, logs, dependencies, and the machine's configuration still matter. In the described implementation, document vectors and content are held in application memory during use; persistence and production-grade document isolation are not claimed.

6. Current Capabilities

The implemented workflow supports PDF upload, text extraction, embedding generation, FAISS-based retrieval, and question answering using a locally served language model. The application also provides a general chat endpoint and a health endpoint. The project was tested in local development, including successful access to the backend health endpoint and a PDF question-answering interaction.

These statements describe observed functional checks, not a controlled performance evaluation. No numerical claims are made about answer accuracy, retrieval precision, latency, throughput, or superiority over other systems.

7. Limitations

The implementation has several limitations. First, there is no reported benchmark dataset or systematic evaluation of answer correctness and faithfulness. Second, performance can vary with document length, PDF text quality, question phrasing, chunking strategy, and the available compute resources. Third, scanned PDFs may not yield usable text without OCR. Fourth, an in-memory document index may not persist after the process restarts and is not a substitute for robust multi-user document management. Fifth, a local language model may produce unsupported answers even when relevant passages are retrieved. Finally, local execution does not automatically provide production security, access control, auditability, or operational monitoring.

8. Future Work

Future work should evaluate the system using a set of PDFs and questions with known answers. Useful metrics include retrieval precision or recall at k, answer correctness, faithfulness to retrieved passages, response latency, and memory use. Experiments could compare chunk sizes and overlap settings, embedding models, retrieval strategies, and language-model configurations. Additional improvements could include explicit source citations in each answer, OCR for scanned documents, persistent vector indexes, document deletion controls, automated tests, and stronger handling of malformed or very large uploads.

Any performance conclusion should be based on recorded measurements with a documented test setup, rather than assumptions or isolated demonstrations.

9. Conclusion

This paper presented the design and implementation of a local PDF question-answering chatbot using a RAG architecture. The application combines a React frontend, FastAPI backend, PyMuPDF text extraction, Sentence Transformers embeddings, FAISS vector retrieval, and Ollama-based local generation. The project demonstrates the integration of these components into a working document-question-answering workflow. A systematic evaluation is still needed before making quantitative claims about quality or performance. The implementation provides a practical starting point for further study of local, document-grounded AI assistants.

References

1. Lewis, P., et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” Advances in Neural Information Processing Systems, 2020. https://arxiv.org/abs/2005.11401

2. Reimers, N., and Gurevych, I. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” Proceedings of EMNLP-IJCNLP, 2019. https://arxiv.org/abs/1908.10084

3. Johnson, J., Douze, M., and Jégou, H. “Billion-scale similarity search with GPUs.” IEEE Transactions on Big Data, 2019. https://github.com/facebookresearch/faiss

4. FastAPI documentation. https://fastapi.tiangolo.com/

5. PyMuPDF documentation. https://pymupdf.readthedocs.io/

6. Ollama documentation. https://ollama.com/

7. Sentence Transformers documentation. https://www.sbert.net/

Publication and Reproducibility Note

This document is a research-style technical report describing an implemented project. It has not been peer reviewed and should not be represented as a formally published or accepted academic paper. No experimental results have been fabricated. Before submitting to a journal, conference, or preprint server, verify the venue's current submission requirements, add reproducibility details and any genuine measured results, and ensure all authorship and source attributions are correct.

Project repository: https://github.com/MAJJARIBHAIRESH/local-ai-chatbot-pdf-rag